```cpp
// LAB CODE: GOOD MED
// JASON JOHNSON



#pragma once
#include <iostream>


using namespace std;

class FeetInches
{
private:
    int feet;        // To hold a number of feet
    int inches;      // To hold a number of inches
    void simplify() {

        feet += inches;
        inches = inches % 12;

        if (inches < 0) {
            feet--;
            inches += 12;
        }
    }
        // Defined in FeetInches.cpp
public:
    // Constructor
    FeetInches() {
        inches = 0;
        feet = 0;
    }

    FeetInches(int f = 0, int i = 0);

    // Mutator functions
    void setFeet(int f)
    {
        feet = f;
    }

    void setInches(int i) {
        inches = i;
    }


    // Accessor functions
    int getFeet() const
```

```
50      {
51          return feet;
52      }
53
54      int getInches() const
55      {
56          return inches;
57      }
58
59      // Overloaded operator functions
60      FeetInches operator + (const FeetInches& second)const {
61          FeetInches temp;
62
63          temp.feet = feet + second.feet;
64          temp.inches = inches + second.inches;
65
66          temp.simplify();
67
68          return temp;
69
70
71      }
72          // Overloaded +
73      FeetInches operator - (const FeetInches& second)const;
74
75          // Overloaded -
76      FeetInches operator ++ () {
77          return ++inches;
78      }
79
80          // Prefix ++
81      FeetInches operator ++ (int) {
82          return inches++;
83
84      }
85          // Postfix ++
86      bool operator > (const FeetInches& second) {
87
88          return feet > second.feet;
89      }
90
91          // Overloaded >
92      bool operator < (const FeetInches& second) {
93
94          return feet < second.feet;
95      }
96
97          // Overloaded <
98      bool operator == (const FeetInches& second) {
```

```
 99          return inches == second.inches;
100      }
101
102
103          // Overloaded ==
104
105      // Friends
106      friend ostream& operator << (ostream& stream, const FeetInches&
           first) {
107
108          stream << "Inches: " << first.inches << endl;
109          stream << "Feet: " << first.feet << endl;
110
111
112      }
113
114
115
116      friend istream& operator >> (istream& stream, FeetInches& second) {
117
118          stream >> second.inches;
119          stream >> second.feet;
120
121      }
122
123
124
125
126
127  };
128
129
130
131
132
133  void FeetInches::simplify() {
134
135      feet += inches;
136      inches = inches % 12;
137
138      if (inches < 0) {
139          feet--;
140          inches += 12;
141      }
142  }
143
144
145  FeetInches::FeetInches(int f = 0, int i = 0) {
146      feet = f;
```

```
147     inches = i;
148
149     simplify();
150
151     }
152
153     // Overloaded operator functions
154     FeetInches operator + (const FeetInches& second)const {
155         FeetInches temp;
156
157         temp.feet = feet + second.feet;
158         temp.inches = inches + second.inches;
159
160         temp.simplify();
161
162         return temp;
163
164
165     }
166     // Overloaded +
167     FeetInches operator - (const FeetInches& second)const {
168
169         FeetInches temp;
170
171         temp.feet = feet - second.feet;
172         temp.inches = inches - second.inches;
173
174         temp.simplify();
175
176         return temp;
177
178
179     }
180
181     // Overloaded -
182     FeetInches operator ++ () {
183         return ++inches;
184     }
185
186     // Prefix ++
187     FeetInches operator ++ (int) {
188         return inches++;
189
190     }
191     // Postfix ++
192     bool operator > (const FeetInches& second) {
193
194         return feet > second.feet;
195     }
```

```
196
197     // Overloaded >
198     bool operator < (const FeetInches& second) {
199
200         return feet < second.feet;
201     }
202
203     // Overloaded <
204     bool operator == (const FeetInches& second) {
205         return inches == second.inches;
206     }
207
208
209     // Overloaded ==
210
211 // Friends
212     friend ostream& operator << (ostream& stream, const FeetInches&
        first) {
213
214         stream << "Inches: " << first.inches << endl;
215         stream << "Feet: " << first.feet << endl;
216
217
218     }
219
220
221
222     friend istream& operator >> (istream& stream, FeetInches& second) {
223
224         stream >> second.inches;
225         stream >> second.feet;
226
227     }
228
229
230
231
232
233
234
235
236
237
238
239
240
241 // This program demonstrates the << and >> operators,
242 // overloaded to work with the FeetInches class.
243
```

```
244 int main()
245 {
246     FeetInches first, second;  // Define two objects.
247
248     // Get a distance for the first object.
249     cout << "Enter a distance in feet and inches.\n";
250     cin >> first;
251
252     // Get a distance for the second object.
253     cout << "Enter another distance in feet and inches.\n";
254     cin >> second;
255
256     // Display the values in the objects.
257     cout << "\nThe values you entered are:\n\n";
258
259     cout << first << " and " << second << endl << endl;
260
261     if (first > second)
262     {
263         cout << "First is longer than the second.\n\n";
264     }
265     else
266     {
267         cout << "The second is longer than the first.\n\n";
268     }
269
270
271     if (first < second)
272     {
273         cout << "The Second is longer than the second.\n\n";
274     }
275     else
276     {
277         cout << "The First is longer than the first.\n\n";
278     }
279
280     if (first == second)
281     {
282         cout << "Both lengths are the same\n\n";
283     }
284     else
285     {
286         cout << "length are not the same.\n\n";
287
288     }
289
290
291     cout << "Adding first to second " << first + second << endl << endl;;
292
```

```
293     cout << "Subtracting first from second " << first - second << endl <<
          endl;
294
295     cout << "Incrementing first " << ++first << endl << endl;
296
297     cout << "Incrementing second " << ++second << endl << endl;
298
299     cout << "Incrementing first " << first++ << endl << endl;
300
301     cout << "Incrementing second " << second++ << endl << endl;
302
303
304
305     return 0;
306 }
307
308 //================ = Output ====================== =
309 //
310 //Enter a distance in feet and inches.
311 //Feet: 1
312 //Inches : 13
313 //Enter another distance in feet and inches.
314 //Feet : 1
315 //Inches : 14
316 //
317 //The values you entered are :
318 //
319 //2 feet, 1 inches and 2 feet, 2 inches
320 //
321 //The second is longer than the first.
322 //
323 //The Second is longer than the second.
324 //
325 //length are not the same.
326 //
327 //Adding first to second 4 feet, 3 inches
328 //
329 //Subtracting first from second 0 feet, 1 inches
330 //
331 //Incrementing first 2 feet, 2 inches
332 //
333 //Incrementing second 2 feet, 3 inches
334 //
335 //Incrementing first 2 feet, 3 inches
336 //
337 //Incrementing second 2 feet, 4 inches
338 //
339 //
340 //C : \Users\med\source\repos\sample\x64\Debug\sample.exe(process 32508)
```

```
      exited with code 0.
341 //To automatically close the console when debugging stops, enable Tools-  ↵
      >Options->Debugging->Automatically close the console when debugging      ↵
      stops.
342 //Press any key to close this window . . .
343
344
```